Experiment No. 09

Title: Reverse Engineering on existing project

Batch: B3 **Roll No.: 16014225801** **Experiment No: 09**

Aim: To analyze an existing open-source software project hosted on GitHub, understand its architecture and source code, identify its software components, and recover design artifacts such as UML diagrams through reverse engineering techniques.

Resources needed: Git, GitHub Account,Visual Studio Code / IntelliJ IDEA / Eclipse, Java JDK / Python (depending on the selected project), Draw.io / StarUML / PlantUML,Internet Connection

---

Theory:

Reverse Engineering is the process of analyzing an existing software system to understand its design, architecture, implementation, and functionality without referring to its original design documents.

Unlike Forward Engineering, which transforms requirements into software, Reverse Engineering starts with the completed software and works backward to recover its design and documentation.

Software reverse engineering is widely used in:

- Software maintenance
- Legacy system modernization
- Security analysis
- Program comprehension
- Software documentation
- Cloud migration
- API understanding

Typical activities include:

- Understanding folder structure
- Identifying software modules
- Identifying packages and classes
- Recovering UML diagrams
- Understanding database interaction
- Identifying APIs
- Understanding deployment architecture

Objectives

1. Understand the concept of software reverse engineering.
2. Analyze the structure of an unfamiliar software project.
3. Identify modules, packages, classes, and their relationships.
4. Recover software architecture from source code.
5. Generate UML diagrams from an existing application.
6. Understand software documentation prepared through reverse engineering.

Procedure:

Step 1: Select an Open-Source GitHub Repository

Step 2: Clone the Repository

Step 3: Open the Observe:

- Folder structure
- Packages
- Configuration files
- README
- Build files
- Dependencies

Step 4: Build and Execute

Step 5: Analyze Project Structure

Step 6: Identify Software Modules

Step 8: Recover any two UML Diagrams

Step 9: Prepare Documentation

- Project objective
- Software architecture
- Modules
- Technologies used
- UML diagrams
- Observations
- Conclusion

Suggested GitHub Projects

No.	Repository	Technology	Difficulty	GitHub Link
1	Spring PetClinic	Spring Boot	Easy	https://github.com/spring-projects/spring-petclinic
2	Spring PetClinic REST	Spring Boot REST	Medium	https://github.com/spring-petclinic/spring-petclinic-rest
3	Docker Voting App	Docker	Easy	https://github.com/dockersamples/example-voting-app
4	Podinfo	Kubernetes	Easy	https://github.com/stefanprodan/podinfo
5	FastAPI Full Stack Template	Python	Medium	https://github.com/fastapi/full-stack-fastapi-template
6	FastAPI RealWorld	Python	Medium	https://github.com/nsidnev/fastapi-realworld-example-app
7	Google Online Boutique	Kubernetes	Advanced	https://github.com/GoogleCloudPlatform/microservices-demo
8	Sock Shop	Docker/Kubernetes	Medium	https://github.com/microservices-demo/microservices-demo

9	JHipster Sample App	Spring Boot	Advanced	https://github.com/jhipster/jhipster-sample-app
10	Langflow	AI Workflow	Advanced	https://github.com/langflow-ai/langflow

---

Outputs:
Repo: https://github.com/bhumitmaru/Whistle-Counter

Project objective
Whistle Counter is a browser-based application that automatically detects and counts pressure-cooker whistles using the device microphone. The user sets a target number of whistles (1–20), starts listening, and the app keeps track of the count in real time, triggering an audible alarm when the target is reached. All audio processing happens locally in the browser; no audio is uploaded, and no account is required.

Software architecture
The project follows a classic component-based front-end architecture with a clear separation between presentation (React components) and behavior (custom React hooks). It is a client-only SPA — there is no backend server, no database, and no persistence layer; all state lives in the browser.

Layered view:
1. Entry layer
— main.tsx boots React, App.tsx wraps the app in BrowserRouter, AppFrame.tsx mounts global UI providers (Tooltip, Toaster, Sonner), AppRoutes.tsx defines the route table.
2. Routing / layout layer
— SiteLayout.tsx is the shell (header, nav, footer, <Outlet />) shared by all routes.
3. Page layer
— pages/Index.tsx (counter), HowItWorks.tsx, About.tsx, FAQ.tsx, NotFound.tsx.
4. Feature component layer
— WhistleCounter.tsx (count display), ControlPanel.tsx (buttons/steppers/toggles), AudioVisualizer.tsx (live audio bars), Seo.tsx (meta tags).
5. Behavioral hook layer
— useWhistleDetector.ts (microphone → FFT → whistle detection) and useAlarm.ts (alarm state + Web Audio tone generation). Hooks own all impure/stateful logic; components stay declarative.
6. UI primitives layer
— components/ui/ (Button, Switch, Tooltip, Toaster, Toast, Sonner) — Radix-based shadcn/ui primitives.
7. Utility layer
— lib/utils.ts (cn() class-merge helper).
8. Build / config layer
— Vite, TypeScript, Tailwind, PostCSS, ESLint; scripts/prerender.mjs pre-renders static HTML per route at build time (the dist/ folder contains the prerendered output).

Modules

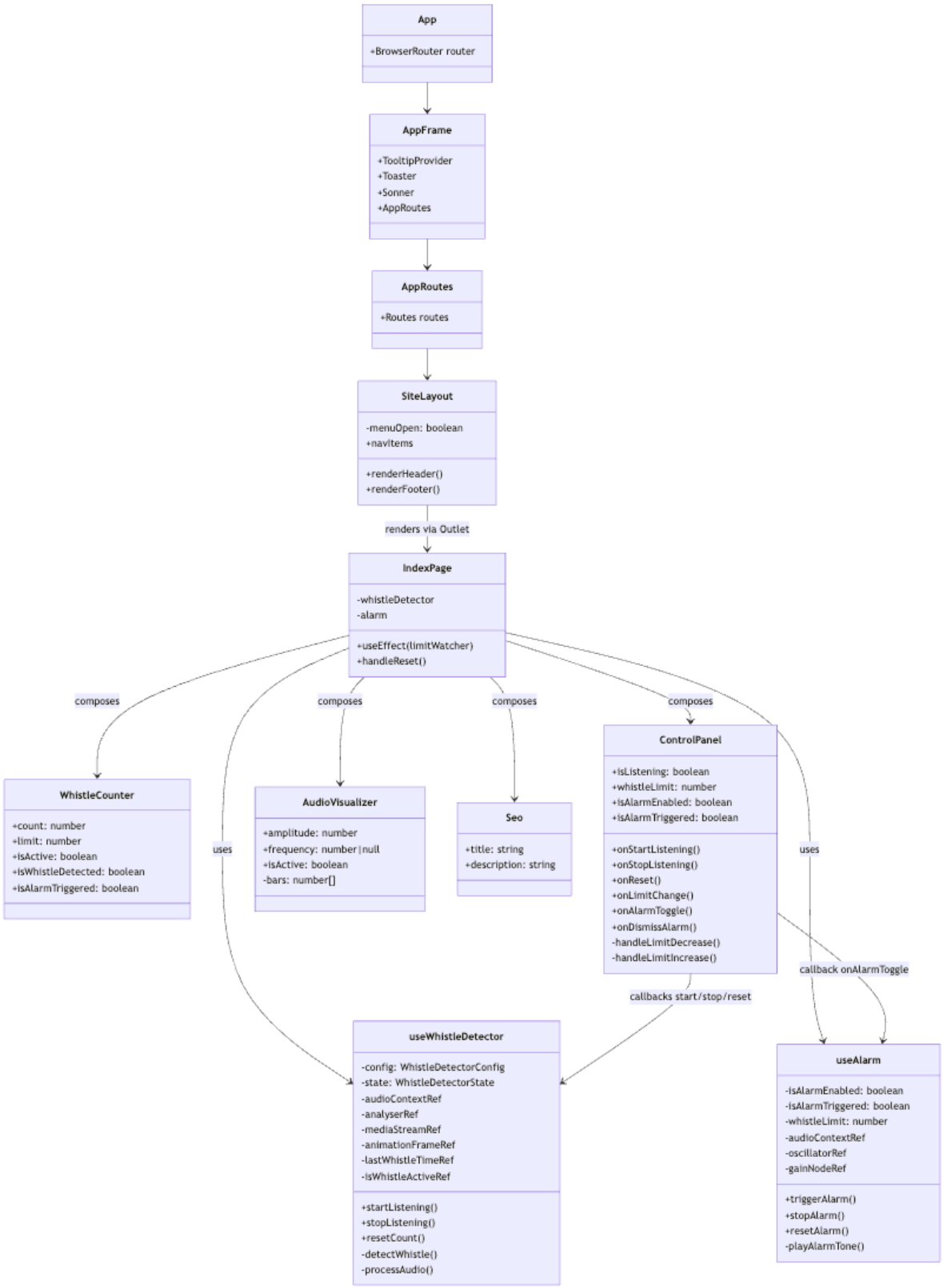
SR NO	MODULE	PATH	RESPONSIBILITY
M1	Bootstrap	main.tsx, App.tsx, AppFrame.tsx	React mount, router, global providers
M2	Routing	AppRoutes.tsx	URL → page mapping
M3	Layout	components/SiteLayout.tsx	Header, nav, footer, scroll handling
M4	Whistle Detection	hooks/useWhistleDetector.ts	Microphone capture, FFT analysis, frequency-band energy detection, cooldown state machine, count increment
M5	Alarm	hooks/useAlarm.ts	Limit state, alarm enable toggle, Web Audio oscillator tone pattern, stop/reset
M6	Counter UI	components/WhistleCounter.tsx	Visual count display, target indicator, status badge, animations
M7	Control Panel	components/ControlPanel.tsx	Start/stop button, limit stepper (1–20), alarm toggle, reset, dismiss alarm
M8	Audio Visualizer	components/AudioVisualizer.tsx	Live amplitude bar chart + frequency readout
M9	SEO	components/Seo.tsx	Document title / meta description per page
M10	UI Primitives	components/ui/*	Radix-based shadcn building blocks
M11	Pages	pages/*	Static content pages (How it works, About, FAQ, 404) + Index composition
M12	Build/Tooling	vite.config.ts, tailwind.config.ts, postcss.config.js, eslint.config.js, tsconfig*.json, scripts/prerender.mjs	Build, lint, type-check, prerender static HTML

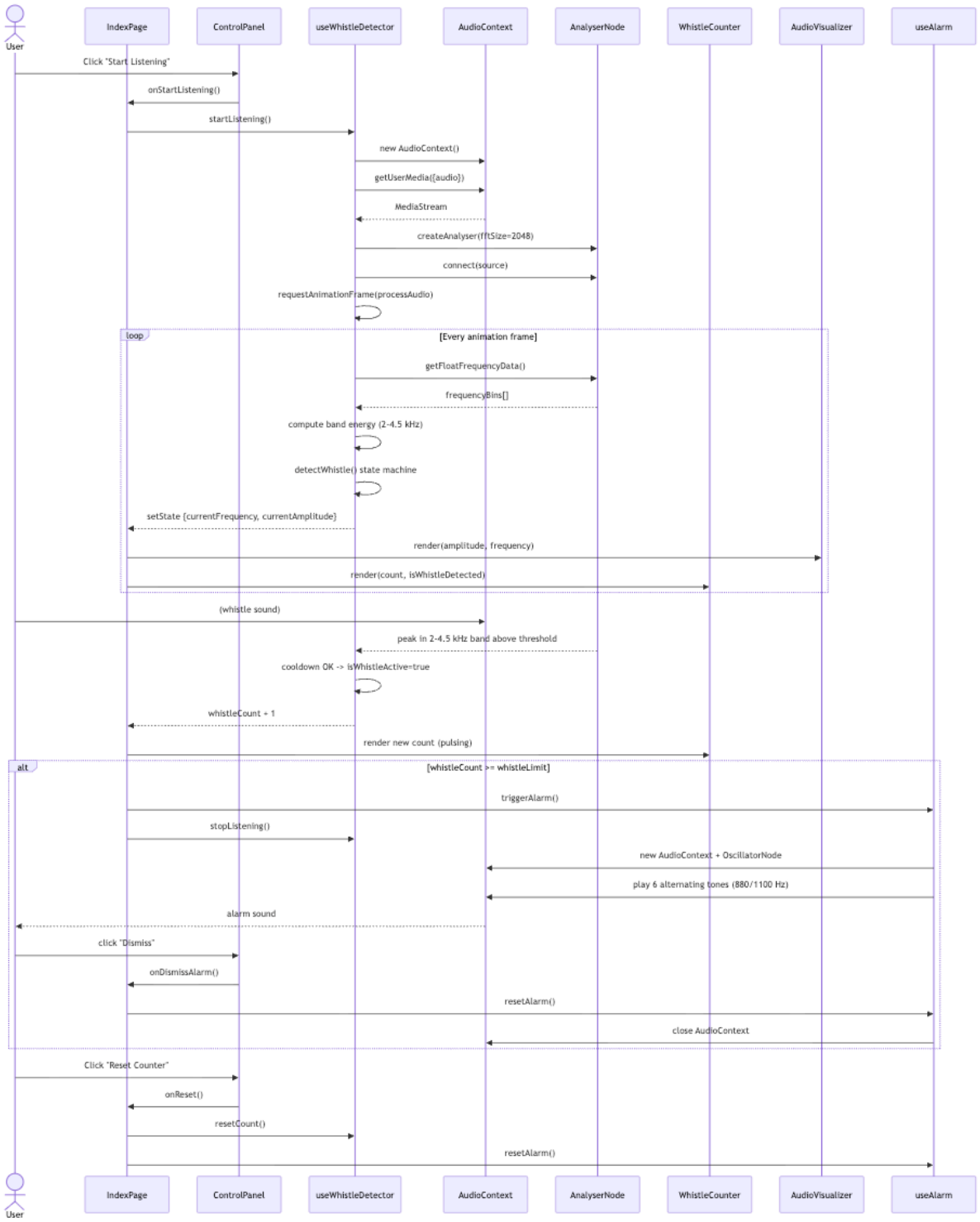
Technologies used

Language: TypeScript (strict, project-referenced config)
UI Library: React 18
Build Tool: Vite 6 (with @vitejs/plugin-react-swc — SWC-based fast refresh)
Styling: Tailwind CSS 3 + tailwindcss-animate + tailwind-merge + clsx + class-variance-authority
Routing: React Router DOM 7
Component Primitives: Radix UI (react-slot, react-switch, react-toast, react-tooltip) via shadcn/ui
Icons: lucide-react
Notifications: Sonner + custom toast hook
Audio: Web Audio API (AudioContext, AnalyserNode, OscillatorNode, GainNode) + navigator.mediaDevices.getUserMedia
Package Manager: npm (lockfile) or Bun (alt lockfile)
Linting: ESLint 9 (flat config) + typescript-eslint + eslint-plugin-react-hooks + eslint-plugin-react-refresh

Deployment hints: public/_redirects and prerendered dist/ indicate static hosting (likely Netlify).

- UML diagrams





Observations & Conclusion

The Whistle-Counter project is a well-structured, single-purpose client-side React + TypeScript SPA that solves a narrow but real problem: counting pressure-cooker whistles without constant attention.

Questions:

1. Why is GitHub a useful source for reverse engineering exercises?

GitHub is a useful source for reverse engineering exercises because it provides access to a large collection of open-source projects, source code, documentation, and software implementations. By studying existing projects, developers can understand the system's architecture, algorithms, design patterns, and functionality. It also allows learners to compare different implementations, identify dependencies, and understand how software components interact.

2. How does reverse engineering differ from forward engineering?

Reverse engineering involves analyzing an existing software system to understand its design, structure, and functionality, often by examining its source code or behavior. In contrast, forward engineering involves developing a software system from requirements and design specifications. Reverse engineering moves from the implementation toward understanding the design, whereas forward engineering moves from requirements and design toward implementation.

Conclusion:

The experiment demonstrated how reverse engineering helps analyze an existing software project, identify its architecture, modules, and technologies, and recover UML diagrams. The Whistle-Counter project illustrated the importance of understanding software structure and documentation.

Grade: AA / AB / BB / BC / CC / CD /DD

Signature of faculty in-charge with date

References:

Books/ Journals/ Websites:

1. Roger S. Pressman, Software Engineering: A practitioners Approach, 7th Edition, McGraw Hill, 2010
2. https://www.sharelatex.com/
3. Atlassian Documentation – JIRA Software Guide
4. Trello Help Center – trello.com/guide